

CCTYPE_REFERENCE

Core Definition

<cctype> is the C++ wrapper for C's **<ctype.h>**: single-byte character classification (**isalpha**, **isdigit**, ...) and case conversion (**tolower**, **toupper**). Every function takes an **int** whose value must be representable as **unsigned char** or equal **EOF** — passing a plain **char** directly is the single most common source of undefined behavior in C++ text code.

#CPP #CCTYPE #CTYPE #CHARACTER #CLASSIFICATION #TOLOWER #ISALPHA #LOCALE

COMPLETE CCTYPE QUICK REFERENCE

FUNCTIONS — Target | Operation | Output

```
// =====
// CLASSIFICATION – all take int, all return int (0 = false, non-zero = true)
// =====
std::isalnum(ch)           // int | Letter OR digit           | isalpha || isdigit
std::isalpha(ch)          // int | Letter               | isupper || islower (+ locale extras)
std::islower(ch)          // int | Lowercase letter      | a-z in the C locale
std::isupper(ch)          // int | Uppercase letter      | A-Z in the C locale
std::isdigit(ch)          // int | Decimal digit        | 0-9 ONLY, in every locale
std::isxdigit(ch)         // int | Hexadecimal digit    | 0-9 A-F a-f
std::isspace(ch)          // int | Whitespace           | space \t \n \v \f \r
std::isblank(ch)          // int | Space or tab          | C++11; space and \t only
std::ispunct(ch)          // int | Printable, not alnum/spc | !"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
std::isprint(ch)          // int | Printable incl. space   | 0x20-0x7E in the C locale
std::isgraph(ch)          // int | Printable, NOT space    | isprint && !isspace
std::iscntrl(ch)          // int | Control character      | 0x00-0x1F and 0x7F in the C locale

// =====
// CONVERSION
// =====
std::tolower(ch)          // int | To lowercase         | Returns int; unchanged if not uppercase
std::toupper(ch)          // int | To uppercase          | Returns int; unchanged if not lowercase

// =====
// THE CONTRACT – read this before using any of the above
// =====
// The argument must be:
//   * representable as unsigned char  (0 .. 255 on a typical platform), OR
//   * equal to EOF                    (typically -1)
// Anything else is UNDEFINED BEHAVIOR. Because `char` is signed on most
// platforms (x86/x64 Linux, Windows, macOS), any byte >= 0x80 becomes a
// negative int when passed directly – outside the contract.
//
//   std::isalpha(c)                // WRONG when char is signed
//   std::isalpha(static_cast<unsigned char>(c)) // CORRECT, always
```

WIDE-CHARACTER COUNTERPARTS — `<cwctype>`

```
std::iswalnum(wc) / iswalpha / iswlower / iswupper / iswdigit / iswxdigit
std::iswspace(wc) / iswblank / iswpunct / iswprint / iswgraph / iswcntrl
std::towlower(wc) / std::towupper(wc)
// All take wint_t; WEOF is the sentinel.
// No signedness trap – wchar_t promotes cleanly.

// Extensible, locale-driven classification (rarely needed):
std::wctype("alpha")           // string | Look up a class      | Returns wctype_t
std::iswctype(wc, desc)        // wc+desc | Test that class    | Returns int
std::wctrans("tolower")        // string | Look up a mapping  | Returns wctrans_t
std::towctrans(wc, desc)       // wc+desc | Apply that mapping  | Returns wint_t
```

LOCALE-AWARE C++ ALTERNATIVES — `<locale>`

```
// Templated, type-safe, explicit about locale – no signedness trap.
std::isalpha(ch, loc)           // charT + locale | Classify      | Returns bool
std::isdigit(ch, loc) / isspace / isupper / islower / ispunct / isalnum /
std::isxdigit(ch, loc) / isprint / isgraph / iscntrl
std::tolower(ch, loc)           // charT + locale | To lower     | Returns charT
std::toupper(ch, loc)          // charT + locale | To upper     | Returns charT

// The underlying facets:
std::use_facet<std::ctype<char>>(loc).is(std::ctype_base::alpha, ch)
std::use_facet<std::ctype<char>>(loc).tolower(ch)
std::ctype_base::alpha / digit / space / upper / lower / punct / xdigit
std::ctype_base::alnum / graph / print / cntrl
```

CLASSIFICATION MATRIX — the C ("POSIX") locale

Character	alnum	alpha	lower	upper	digit	xdigit	space	blank	punct	print	graph	cntrl
'a'-'f'	X	X	X				X				X	X
'g'-'z'	X	X	X									
'A'-'F'	X	X		X			X				X	X
'G'-'Z'	X	X		X								
'0'-'9'	X				X		X				X	X
'!' '#' '\$' ...									X	X	X	X
' ' (space)								X	X		X	
'\t'							X	X				X
'\n' '\v' '\f' '\r'							X					X
0x00-0x08, 0x0E-0x1F												X
0x7F (DEL)												X

Every row above is guaranteed in the C locale. Other locales may add characters to `alpha`, `punct`, `print`, `upper` and `lower` — but **never** to `digit` or `xdigit`, which are fixed to ASCII by the standard.

COMMON PATTERNS & EXAMPLES

The Cast — Always

```
#include <cctype>

// WRONG – undefined behavior for any byte >= 0x80 when char is signed
bool badIsAlpha(char c) { return std::isalpha(c); }

// RIGHT
bool isAlpha(char c) { return std::isalpha(static_cast<unsigned char>(c)); }

// A small helper is worth writing once and reusing everywhere:
inline unsigned char uc(char c) { return static_cast<unsigned char>(c); }

if (std::isspace(uc(c))) { /* ... */ }
```

The failure is not theoretical: 'é' in a Latin-1 string is 0xE9, which becomes -23 as a signed `char`. Most implementations index a lookup table by that value, reading memory before the table's start.

Case Conversion on a String

```
#include <algorithm>
#include <cctype>
#include <string>

std::string toLower(std::string s) {
    std::transform(s.begin(), s.end(), s.begin(),
        [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
    return s;
}
```

Two casts matter here. The lambda parameter is `unsigned char`, which satisfies the input contract. The result is cast back to `char`, since `tolower` returns `int` and assigning it straight into a `char` container is a narrowing conversion.

Case-Insensitive Comparison

```
#include <cctype>
#include <string>
#include <algorithm>

bool iequals(std::string_view a, std::string_view b) {
    return a.size() == b.size() &&
        std::equal(a.begin(), a.end(), b.begin(),
            [](unsigned char x, unsigned char y) {
                return std::tolower(x) == std::tolower(y);
            });
}
```

This is an ASCII-correct comparison, not a Unicode-correct one. Turkish dotless ı, German ß → SS, and Greek final sigma all break it. For real case folding use ICU.

Trimming Whitespace

```
#include <cctype>
#include <string>
#include <algorithm>

std::string trim(const std::string& s) {
    auto notSpace = [](unsigned char c) { return !std::isspace(c); };
    auto begin = std::find_if(s.begin(), s.end(), notSpace);
    auto end = std::find_if(s.rbegin(), s.rend(), notSpace).base();
    return (begin < end) ? std::string(begin, end) : std::string();
}
```

Validating Input

```
#include <cctype>
#include <string_view>
#include <algorithm>

bool allDigits(std::string_view s) {
    return !s.empty() && std::all_of(s.begin(), s.end(),
        [](unsigned char c) { return std::isdigit(c); });
}

bool isIdentifier(std::string_view s) {
    if (s.empty()) return false;
    if (!std::isalpha(static_cast<unsigned char>(s[0])) && s[0] != '_' ) return false;
    return std::all_of(s.begin() + 1, s.end(), [](unsigned char c) {
        return std::isalnum(c) || c == '_';
    });
}
```

Counting Character Categories

```
#include <cctype>
#include <iostream>

int main() {
    int letters = 0, digits = 0, spaces = 0, punct = 0, other = 0;
    int c;

    while ((c = std::cin.get()) != EOF) { // get() returns int – EOF is legal input
        if (std::isalpha(c)) ++letters;
        else if (std::isdigit(c)) ++digits;
        else if (std::isspace(c)) ++spaces;
        else if (std::ispunct(c)) ++punct;
        else ++other;
    }
    std::cout << letters << ' ' << digits << ' ' << spaces << ' ' << punct << '\n';
}
```

No cast is needed here: `istream::get()` already returns an `int` in the valid range, or `EOF`.

Hex Digit to Value

```
#include <cctype>
#include <optional>

std::optional<int> hexValue(char c) {
    unsigned char u = static_cast<unsigned char>(c);
    if (!std::isxdigit(u)) return std::nullopt;
    if (std::isdigit(u)) return u - '0';
    return std::tolower(u) - 'a' + 10;
}
```

A Simple Tokenizer

```
#include <cctype>
#include <string>
#include <vector>

std::vector<std::string> words(const std::string& text) {
    std::vector<std::string> out;
    std::string cur;
    for (unsigned char c : text) {           // Iterating as unsigned char does the cast for you
        if (std::isalnum(c)) {
            cur += static_cast<char>(c);
        } else if (!cur.empty()) {
            out.push_back(std::move(cur));
            cur.clear();
        }
    }
    if (!cur.empty()) out.push_back(std::move(cur));
    return out;
}
```

Locale-Aware Classification

```
#include <locale>
#include <iostream>

int main() {
    std::locale loc("");                      // The user's environment locale

    char c = 'A';
    bool alpha = std::isalpha(c, loc);        // Templated form – no cast needed
    char low = std::tolower(c, loc);          // Returns char, not int

    // Direct facet access, for classifying a whole buffer at once:
    const auto& ct = std::use_facet<std::ctype<char>>(loc);
    std::string s = "Hello, World";
    ct.tolower(s.data(), s.data() + s.size()); // In-place, one call
    std::cout << s << '\n';
}
```

std::locale("") reads the environment and can throw `std::runtime_error` if the named locale is unavailable. Wrap it or fall back to `std::locale::classic()`.

Avoiding the Trap Structurally

```
// Give yourself an interface that cannot be misused:
namespace chr {
    inline bool alpha(char c) { return std::isalpha(static_cast<unsigned char>(c)); }
    inline bool digit(char c) { return std::isdigit(static_cast<unsigned char>(c)); }
    inline bool space(char c) { return std::isspace(static_cast<unsigned char>(c)); }
    inline char lower(char c) { return static_cast<char>(std::tolower(static_cast<unsigned char>(c))); }
    inline char upper(char c) { return static_cast<char>(std::toupper(static_cast<unsigned char>(c))); }
}
// Then the rest of the codebase writes chr::alpha(c) and the cast never appears again.
```

IMPORTANT CONCEPTS

Why the Signature Is `int`

These functions predate `unsigned char` conventions and were designed to accept the result of `getchar()`, which returns either a character value in `unsigned char` range or `EOF`. That is why the domain is "`unsigned char` range or `EOF`" rather than simply "a character" — and why a signed `char` argument falls outside it.

Typical Implementations Make the Bug Silent

A common implementation is a table lookup: `__ctype_table[c]`. With `c == -23` this reads 23 bytes before the table. Many libraries deliberately allocate padding there so the read "works", which means the bug produces wrong answers rather than a crash — and only on non-ASCII input, which test suites often lack.

`isdigit` Is Always ASCII

The standard fixes `digit` to `'0' - '9'` and `xdigit` to those plus `A - F / a - f` in every locale. Arabic-Indic digits are never `isdigit`. This makes `isdigit` safe for parsing wire formats, unlike `isalpha` or `ispunct`.

`tolower` Returns `int`

Assigning the result directly into a `char` narrows. In a lambda passed to `std::transform` over a `std::string`, that narrowing is silent unless warnings are on. Cast the result explicitly.

One Byte Is Not One Character

`<cctype>` operates on single bytes. In UTF-8 text, any code point above `U+007F` spans multiple bytes, and no individual byte of such a sequence is meaningfully "alphabetic". Applying `toupper` byte by byte to UTF-8 leaves multi-byte characters unchanged at best and corrupts them at worst. For text beyond ASCII, use ICU — `<cctype>`, `<cwctype>` and `<locale>` all fall short of real Unicode semantics.

Locale Changes Behavior at Runtime

`std::setlocale(LC_CTYPE, ...)` changes what the non-templated functions report — globally, for every thread. Library code that classifies characters should either use the templated `<locale>` forms with an explicit locale, or restrict itself to `isdigit` / `isxdigit`, which the standard pins to ASCII.

`isblank` Is C++11

It is the only addition to the classification set since C89. If you are targeting an older toolchain, test for `' '` and `'\t'` directly.

Prefer `<charconv>` for Parsing Numbers

Hand-rolling integer parsing on top of `isdigit` is a common exercise but rarely the right production answer. `std::from_chars` (C++17) is locale-independent, non-allocating, checks overflow, and reports where parsing stopped.

SAFETY CHEAT SHEET

Written as	Verdict
<code>isalpha(c)</code> <code>// char c</code>	UB for bytes $\geq 0x80$
<code>isalpha((unsigned char)c)</code>	Correct
<code>isalpha(uc(c))</code>	Correct, and readable
<code>for (unsigned char c : str) isalpha(c)</code>	Correct – the loop does the cast
<code>isalpha(std::cin.get())</code>	Correct – <code>get()</code> returns <code>int/EOF</code>
<code>isalpha(c, loc)</code> <code>// <locale></code>	Correct, and locale-explicit
<code>tolower(c)</code> assigned to <code>char</code>	Narrowing – cast the result
<code>toupper</code> on a UTF-8 byte	Wrong model – use ICU
<code>isdigit</code> for wire-format parsing	Safe – always ASCII
<code>isalpha</code> for wire-format parsing	Locale-dependent – avoid

BEST PRACTICES

1. Always pass `static_cast<unsigned char>(c)` — no exceptions, no "it works on my machine"
2. Wrap the family once in a small namespace so the cast appears in exactly one place
3. Iterate as `unsigned char` (`for (unsigned char c : s)`) and the cast disappears
4. Cast `tolower`/`toupper` results back to `char` before storing
5. Use the `<locale>` templated forms when locale correctness matters — they take `char` safely
6. Rely on `isdigit`/`isxdigit` for protocol parsing; treat the rest as locale-dependent
7. Do not apply these to UTF-8 bytes and expect character semantics
8. Prefer `std::from_chars` over hand-written `isdigit` loops for numbers
9. Remember `std::setlocale` is global and process-wide — library code should not call it
10. Enable `-Wconversion` to catch the silent `int` $\rightarrow$ `char` narrowing
11. Test with non-ASCII input — the signed-char bug is invisible on pure ASCII

RELATED HEADERS

<code>#include <cctype></code>	<code>// isalpha, isdigit, tolower ... (single-byte)</code>
<code>#include <cwctype></code>	<code>// iswalph, towlower ... (wide characters)</code>
<code>#include <locale></code>	<code>// templated isalpha(c, loc), ctype facet</code>
<code>#include <string></code>	<code>// std::string, char_traits</code>
<code>#include <string_view></code>	<code>// C++17 non-owning views to classify over</code>
<code>#include <algorithm></code>	<code>// transform, all_of, find_if, equal</code>
<code>#include <charconv></code>	<code>// C++17 from_chars – the right way to parse numbers</code>
<code>#include <cstring></code>	<code>// C-string functions this often accompanies</code>

EXTERNAL RESOURCES

- **Null-terminated byte strings — classification:** <https://en.cppreference.com/w/cpp/string/byte>
- **<cctype>:** <https://en.cppreference.com/w/cpp/header/cctype>
- **std::ctype facet:** <https://en.cppreference.com/w/cpp/locale/ctype>
- **cppreference note on the unsigned char requirement:**
<https://en.cppreference.com/w/cpp/string/byte/isalpha>

Standard: C89 base; `isblank` added in C++11; templated `<locale>` forms since C++98 **Last Updated:** September 2026